1. Why Do We Need External Locations?

In real projects, source data often already exists in cloud storage such as **ADLS Gen2**.

Example:

ADLS Gen2
↓
orders.csv

Databricks needs a secure way to access this data.

Hardcoding:

- Storage keys
- Secrets
- Credentials

inside notebooks is not a good production approach.

Unity Catalog provides:

Storage Credential
+
External Location

to govern external storage access.

---

2. Architecture to Remember

The complete flow from this class is:

Azure Storage
↓
Databricks Access Connector
↓
Managed Identity
↓
Storage Credential
↓
External Location
↓

orders.csv

The most important distinction:

Storage Credential
→ HOW Databricks authenticates

External Location
→ WHICH storage path Databricks can access

---

3. What Is a Storage Credential?

A **Storage Credential** is a Unity Catalog object that represents how Databricks authenticates to cloud storage.

In Azure, it can reference the managed identity provided by an **Access Connector for Azure Databricks**.

It does **not** contain your actual business data.

Interview Answer

> A Storage Credential defines the authentication mechanism Databricks uses to access external cloud storage.

---

4. What Is an External Location?

An **External Location** combines:

Cloud Storage Path
+
Storage Credential

Example:

abfss://datax@<storage-account>.dfs.core.windows.net/landing/orders/

The External Location tells Unity Catalog:

Use this credential

↓
to access this path

It can then be used to govern external storage access.

---

5. Storage Credential vs External Location

Storage Credential	External Location
Defines authentication	Defines governed storage path
Answers **how to authenticate**	Answers **where to access**
Can reference managed identity	References a Storage Credential
Can be reused	Represents a particular authorized path

Easy Memory Trick

Credential = WHO / HOW

External Location = WHERE

---

6. Role of the Databricks Access Connector

The **Access Connector for Azure Databricks** provides a managed identity.

That managed identity can be granted access to Azure storage.

Instead of putting this inside notebooks:

Storage Account Key
Secret
Password

we use:

Managed Identity

This provides a cleaner and more secure authentication pattern.

---

7. Azure RBAC and Unity Catalog Are Different Layers

This is one of the most important interview points.

Suppose the managed identity has:

Storage Blob Data Contributor

on the Azure storage account.

Does that automatically mean every Databricks user can read the files?

No.

There are two separate layers:

Azure RBAC
→ Can the managed identity access Azure Storage?

Unity Catalog Permissions
→ Can this Databricks user/job perform the operation?

Both layers must permit the required access.

---

8. Creating the Storage Credential

In the class, the Storage Credential references the Access Connector's managed identity.

Example name:

datax_adls_credential

Verify available credentials:

SHOW STORAGE CREDENTIALS;

Important point:

Storage Credential
≠
Storage Path

It only represents the authentication mechanism.

---

9. Creating the External Location

Example:

```
CREATE EXTERNAL LOCATION datax_orders_location
URL 'abfss://datax@<storage-account>.dfs.core.windows.net/landing/orders/'
WITH (
   STORAGE CREDENTIAL datax_adls_credential
);
```

Relationship:

```
datax_orders_location
      ↓
ADLS Path
      +
datax_adls_credential
```

Important

Creating an External Location does **not**:

- Move the files
- Create a Delta table
- Automatically give every user file access

It registers and governs access to the existing path.

---

10. Useful Commands

List External Locations:

```
SHOW EXTERNAL LOCATIONS;
```

Inspect one:

```
DESCRIBE EXTERNAL LOCATION
datax_orders_location;
```

Inspect permissions:

```
SHOW GRANTS
ON EXTERNAL LOCATION datax_orders_location;
```

These are important commands for both production troubleshooting and interviews.

---

11. Reading Data from External Storage

Once the storage access is configured correctly:

```
external_orders_path = (
    "abfss://datax@<storage-account>.dfs.core.windows.net/"
    "landing/orders/orders.csv"
)
```

Then:

```
external_orders_df = (
    spark.read
    .option("header", "true")
    .schema(orders_schema)
    .csv(external_orders_path)
)
```

The important part is not the CSV read itself.

The important part is that authentication and governance are handled through the storage configuration rather than hardcoded credentials.

---

12. External Location Privileges

Important privileges covered in the class:

`READ FILES`

Read files directly from an External Location.

`WRITE FILES`

Write files directly to an External Location.

`CREATE EXTERNAL TABLE`

Create external tables using the authorized location.

`CREATE EXTERNAL VOLUME`

Create external volumes using the authorized location.

Example:

```
GRANT READ FILES
ON EXTERNAL LOCATION datax_orders_location
TO `datax_engineers`;
```

Least Privilege

If a Data Engineer only needs to read source CSV files:

READ FILES

may be enough.

Do not automatically grant:

WRITE FILES

unless it is required.

---

13. DEV vs PROD Storage Access

A good production design separates environments.

Example:

DEV
→ dev_storage_credential
→ dev_orders_location

PROD
→ prod_storage_credential
→ prod_orders_location

Developers may have read/write access to approved DEV paths.

Production write access may be restricted to:

- Production jobs
- Service principals
- Approved identities

This reduces accidental production changes.

---

14. Troubleshooting External Location Access

Scenario

You created the External Location, but your notebook still cannot read the file.

Troubleshoot layer by layer:

1. Storage Path
2. Storage Credential
3. Access Connector
4. Azure RBAC
5. Networking
6. ACLs if applicable
7. Unity Catalog Permissions
8. File Existence
9. Permission Propagation

Important Interview Approach

Do not recreate everything immediately.

First identify **which layer is failing**.

---

15. Common Mistakes

Mistake 1 — Hardcoding Storage Keys

Avoid storing Azure storage credentials directly inside notebooks.

Prefer managed identity where supported.

Mistake 2 — Confusing Credential and Location

Storage Credential
→ Authentication

External Location
→ Path + Credential

Mistake 3 — Giving Broad Storage Access

Use the narrowest practical:

- Azure scope
- Storage path
- Unity Catalog privilege

Mistake 4 — Assuming Azure RBAC Alone Is Enough

It isn't.

Unity Catalog permissions also matter.

Mistake 5 — Poorly Planned Overlapping External Locations

Overlapping storage paths can create governance and object-management complications.

---

16. Interview Questions

Q1. What is a Storage Credential?

A Unity Catalog object that represents the authentication mechanism used to access cloud storage.

Q2. What is an External Location?

A Unity Catalog object that combines a cloud storage path with a Storage Credential so access to that path can be governed.

Q3. What is the difference between Storage Credential and External Location?

Storage Credential
→ How Databricks authenticates

External Location
→ Which cloud storage path is governed/accessed

Q4. What is the role of the Azure Databricks Access Connector?

It provides a managed identity that Databricks can use to authenticate to Azure resources such as ADLS Gen2.

Q5. Why prefer managed identity over storage account keys?

Because managed identity avoids embedding long-lived storage keys in notebooks and supports centralized identity and access management.

Q6. What is the difference between Azure RBAC and Unity Catalog permissions?

Azure RBAC controls whether the managed identity can access Azure storage.

Unity Catalog controls which Databricks users or workloads can perform governed operations on the external location.

Q7. What privilege is commonly required for direct file reads?

READ FILES

on the External Location, subject to the applicable authorization model.

Q8. Can one Storage Credential be reused?

Yes.

A Storage Credential can be referenced by multiple External Locations, subject to the identity and permission requirements.

Q9. Does creating an External Location create a Delta table?

No.

It registers and governs a cloud storage location.

Tables and volumes are separate objects.

Q10. What would you check if External Location validation fails?

Check:

- Storage URL
- Access Connector
- Managed identity
- Azure RBAC
- Networking
- ACLs where applicable
- External Location configuration

---

17. Scenario-Based Interview Questions

Scenario 1

How would you securely connect Azure Databricks to ADLS Gen2 without hardcoding keys?

Use:

Access Connector
↓
Managed Identity
↓
Azure Storage Permission
↓
Storage Credential
↓
External Location
↓
Required UC Privileges

Scenario 2

Managed identity has Storage Blob Data Contributor, but the user still gets access denied. Why?

Because Azure RBAC is only one layer.

The Databricks user may still be missing Unity Catalog permissions such as `READ FILES`.

Scenario 3

Why use different credentials/locations for DEV and PROD?

To isolate environments and reduce the risk of developers or test workloads modifying production data.

Scenario 4

A Data Engineer only needs to read landing CSV files. Would you give `WRITE FILES`?

No.

Follow least privilege and grant only the required read access.

---

18. Quick Revision

Access Connector
→ Provides Managed Identity

Storage Credential
→ Authentication

External Location
→ Storage Path + Credential

Azure RBAC
→ Azure-side storage access

Unity Catalog
→ Databricks-side governed access

READ FILES
→ Direct file reading

WRITE FILES
→ Direct file writing

Final Takeaway

Storage Credentials define how Databricks authenticates to external storage, while External Locations define which cloud-storage paths are governed and accessible through that credential.